

Laboratorio 2 de Tecnologías de Microprocesamiento

Pablo Ramirez, Braian Arias
Ingeniería Mecatrónica
Universidad Tecnológica – UTEC
Unidad Curricular: Tecnologías de Microprocesamiento
Docentes: Jesús Martínez y Marcelo Díaz

Resumen—En este trabajo se presentan el diseño, implementación y evaluación de diferentes aplicaciones desarrolladas utilizando el microcontrolador ATmega328P mediante lenguaje ensamblador AVR.

El laboratorio integra el manejo de entradas y salidas digitales, direccionamiento de memoria, Stack Pointer, tablas de consulta LUT, temporizadores, interrupciones y comunicación serie USART/UART.

Las aplicaciones desarrolladas incluyen el control de una matriz de LEDs con mensajes desplazables, la generación de señales analógicas mediante un convertidor DAC R-2R, el control de un plotter, la comunicación USART entre dos microcontroladores y la automatización de una puerta de garaje mediante una máquina de estados.

Además de la implementación funcional, se documentan las decisiones tomadas durante el desarrollo, las máquinas de estados empleadas, el uso de registros del ATmega328P y las pruebas realizadas mediante simulación y montaje experimental.

Index Terms—ATmega328P, AVR, ensamblador, USART, UART, máquina de estados, R-2R, LUT, matriz LED, plotter, interrupciones.

I. INTRODUCCIÓN

El microcontrolador ATmega328P pertenece a la arquitectura AVR de 8 bits y dispone de diferentes periféricos que permiten desarrollar aplicaciones de control, adquisición y comunicación.

El objetivo del laboratorio es profundizar en el manejo del ATmega328P mediante programación en lenguaje ensamblador, incorporando el uso del Stack Pointer, direccionamiento de memoria, temporizadores, contadores, interrupciones, tablas de consulta o Look-Up Tables (LUT) y comunicación serie USART/UART.

Para alcanzar este objetivo, se desarrollaron cinco problemas independientes, denominados Problemas A, B, C, D y E. Cada uno de ellos permite utilizar diferentes recursos disponibles en la arquitectura del microcontrolador.

Las soluciones fueron desarrolladas mayormente mediante simulación previa a la implementación física. Como excepción, el Problema C (control del plotter) se abordó directamente sobre hardware, dada la fuerte dependencia del comportamiento mecánico real y del PLC asociado, lo que hacía poco representativa una simulación previa. Las simulaciones se hicieron con el propósito de comprobar el funcionamiento del programa antes de realizar la implementación física.

II. METODOLOGÍA

Para el desarrollo de los diferentes problemas se utilizó una metodología dividida en las siguientes etapas:

1. Análisis del problema.
2. Identificación de entradas y salidas.
3. Diseño del algoritmo de funcionamiento.
4. Diseño de la máquina de estados cuando corresponda.
5. Configuración de los registros del ATmega328P.
6. Programación en lenguaje ensamblador AVR.
7. Simulación del circuito.
8. Corrección de errores.
9. Implementación experimental.
10. Registro de las modificaciones mediante GitHub.

Para cada aplicación se buscó utilizar directamente los recursos internos del microcontrolador, trabajando con los registros de propósito general y los registros especiales correspondientes a cada periférico.

III. MATERIALES UTILIZADOS

Para el desarrollo del laboratorio se utilizaron distintos componentes electrónicos, instrumentos de medición y sistemas electromecánicos.

Los principales materiales utilizados se presentan a continuación.

III-A. Materiales generales

El ATmega328P constituye el elemento principal de procesamiento de las diferentes aplicaciones.

La protoboard permite realizar las conexiones experimentales, mientras que los LEDs, resistencias, pulsadores y cables se utilizan para implementar las diferentes entradas y salidas de los sistemas.

IV. PROBLEMA A: MATRIZ DE LEDs

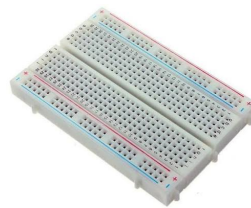
IV-A. Descripción

El Problema A consiste en controlar una matriz de LEDs mediante el microcontrolador ATmega328P.

El sistema debe mostrar un mensaje desplazándose de derecha a izquierda de forma suave y continua, como también 3 figuras

(PCINT14/RESET) PC6	1	28	PC5 (ADC5/SCL/PCINT13)
(PCINT16/RXD) PD0	2	27	PC4 (ADC4/SDA/PCINT12)
(PCINT17/TXD) PD1	3	26	PC3 (ADC3/PCINT11)
(PCINT18/INT0) PD2	4	25	PC2 (ADC2/PCINT10)
(PCINT19/SCK/INT1) PD3	5	24	PC1 (ADC1/PCINT9)
(PCINT20/SCK/INT0) PD4	6	23	PC0 (ADC0/PCINT8)
VCC	7	22	GND
GND	8	21	AREF
(PCINT16/XIAL1/TOSC1) PB6	9	20	AVCC
(PCINT17/XIAL2/TOSC2) PB7	10	19	PB1 (SCK/PCINT7)
(PCINT21/OC0B/PT1) PD7	11	18	PB4 (ADSC/PCINT4)
(PCINT22/OC0A/AIN0) PD6	12	17	PB5 (ADSC/OC2A/PCINT3)
(PCINT21/AIN1) PD7	13	16	PB2 (SS/OC1B/PCINT2)
(PCINT0/CLK0/ICP1) PB0	14	15	PB1 (OC1A/PCINT1)

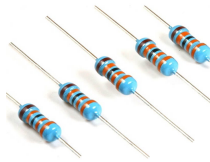
ATmega328P



Protoboard



LED



Resistencias



Pulsadores



Cables

Figura 1. Materiales generales utilizados durante el laboratorio.

La frase seleccionada para la implementación fue: cocos locos! se desplasa de derecha a izquierda pero se tiene que girar la matriz y se ve como se desplaza la palabra hacia abajo

C
O
C
O
S
espacio
L
O
C
O
S
!

Las figuras son cara sonriente, cara sonriente con guiño, corazon :) <3

Además, mediante la comunicación UART, el usuario puede seleccionar entre visualizar el mensaje desplazable o alguna de las figuras almacenadas previamente.

IV-B. Materiales utilizados

Los principales materiales utilizados en este problema fueron:

IV-C. Funcionamiento general

Al iniciar el sistema se configura la matriz de LEDs y el módulo USART.

Posteriormente se transmite hacia el terminal serial un menú con las opciones disponibles.

El usuario selecciona una opción mediante UART. Dependiendo del carácter recibido, el programa ejecuta el mensaje desplazable o una de las figuras almacenadas.

IV-D. Máquina de estados

La Fig. 3 representa el funcionamiento general del sistema.

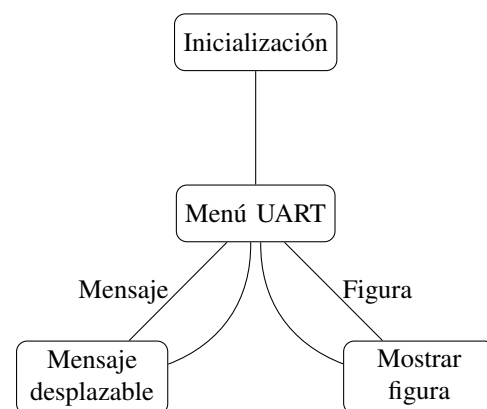


Figura 3. Máquina de estados simplificada del Problema A.

IV-E. Implementación

Para representar los caracteres se almacenan patrones gráficos en memoria.

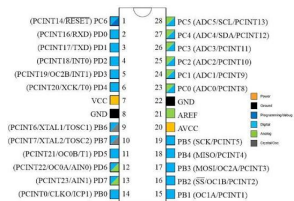
La información correspondiente a cada columna del carácter es leída secuencialmente y enviada hacia la matriz.

El desplazamiento se consigue modificando periódicamente las columnas mostradas.

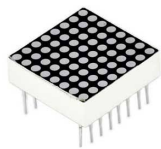
La velocidad del desplazamiento se controla mediante una temporización, permitiendo que el mensaje resulte legible.

IV-F. Código principal

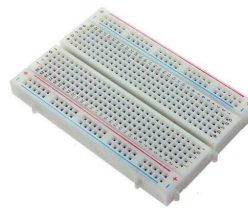
Se utilizó el registro Z (R31:R30) como puntero indirecto para acceder a las tablas de caracteres almacenadas en memoria de programa (Flash). La velocidad de desplazamiento se controla mediante una variable en RAM (velocidad) que ajusta el número de iteraciones del retardo software. Se implementó



ATmega328P



Matriz de LEDs



Protoboard



Cables

Figura 2. Materiales utilizados en el Problema A.

un menú serial que permite seleccionar entre figuras predefinidas (cmd1, cmd2, cmd3) o mostrar una palabra personalizada (cmd-palabra"), además de comandos de ajuste de frecuencia (+/-)

Listing 1. Estructura del programa del Problema A

```

1  .include "m328pdef.inc"
2  .equ VEL_INICIAL = 18
3  .equ VEL_MIN = 4
4  .equ VEL_MAX = 60
5  .dseg
6  .org 0x0100
7  buffer:
8  .byte 8
9  velocidad:
10 .byte 1
11 .cseg
12 .ORG 0x0000
13 .equ F_CPU = 16000000
14 .equ baud = 9600
15 .equ bps = (F_CPU/16/baud)-1
16 RJMP Inicio
17
18 Inicio:
19 ; Inicializar stack
20 ldi r16,HIGH(RAMEND)
21 out SPH,r16
22 ldi r16,LOW(RAMEND)
23 out SPL,r16
24 ;
25
26 ; como todo es salida para la matriz menos tx y rx
27 ; todo salida y despues defino
28 PORTD: PD0 RX, PD1 TX, PD2-PD7 matriz
29 ldi r16,0b11111100
30 out DDRD,r16
31
32 ; PORTB: PB0-PB5 matriz
33 ldi r16,0b00111111
34 out DDRB,r16
35
36 ; PORTC: PC0-PC3 matriz
37 ldi r16,0b00001111
38 out DDRC,r16
39
40 ; Velocidad inicial
41 ldi r16,VEL_INICIAL
42 sts velocidad,r16
43
44 ; Inicializar UART
45 rcall apagar_matriz
46 rcall uart_init
47
48 main_loop:
49 rcall mostrar_menu
50 ; Esperar y leer carcter
51
52 esperar_comando:
53 rcall uart_rx
54
55 ; Ignorar ENTER
56 cpi r16,0x0D
57 breq esperar_comando
58 cpi r16,0x0A
59 breq esperar_comando
60
61 Comparar:
62 cpi r16,'1' ; poner 1 y enter en el data visual
63 breq cmd1
64 cpi r16,'2'
65 breq cmd2
66 cpi r16,'3'
67 breq cmd3
68 cpi r16,'p'
69 breq cmd_palabra
70 cpi r16,'P' ; por si esta en mayuscula
71 breq cmd_palabra
72 cpi r16,'+'
73 breq cmd_rapido
74 cpi r16,'-'
75 breq cmd_lento
76 rjmp esperar_comando
77
78 cmd1:
79 ldi ZL,LOW(mensaje_sonrisa<<1)
80 ldi ZH,HIGH(mensaje_sonrisa<<1)
81 rcall uart_print ; escribel el mensaje
82 rcall sonrisa ; llama la figura y vuelve a esperar
83 ; comando
84 rjmp esperar_comando

```

V. PROBLEMA B: CONVERTIDOR DAC R-2R

V-A. Descripción

En este problema se implementa un convertidor digital-analógico mediante una red de resistencias R-2R de 8 bits.

El ATmega328P genera valores digitales comprendidos entre 0 y 255.

Estos valores son enviados hacia el convertidor R-2R, obteniéndose una tensión analógica proporcional.

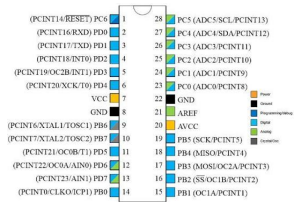
V-B. Materiales utilizados

V-C. Uso de LUT

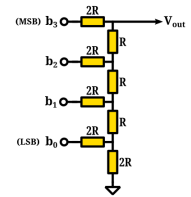
Los valores correspondientes a la señal se almacenan previamente en una tabla de consulta o LUT.

La LUT contiene una secuencia de muestras digitales:

$$x[n] \in [0, 255]$$



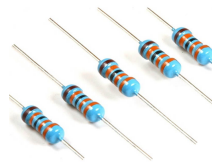
ATmega328P



DAC R-2R



Osciloscopio



Resistencias

Figura 4. Materiales utilizados en el Problema B.

(1)

V-D. Máquina de estados

Cada valor es enviado secuencialmente hacia el puerto conectado al DAC R-2R.

La tensión aproximada puede expresarse como:

$$V_{out} = \frac{D}{255} V_{ref}$$

(2)

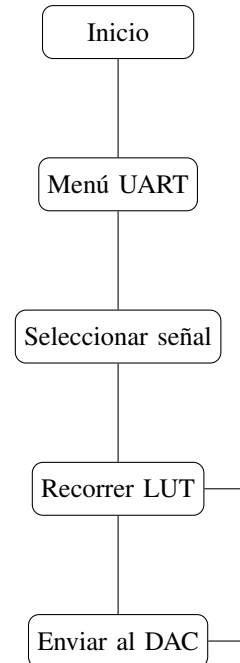


Figura 5. Flujo de funcionamiento del generador de señales.

V-E. Control de frecuencia

donde D representa el valor digital enviado por el micro-controlador.

La frecuencia de la señal generada depende del tiempo transcurrido entre dos muestras consecutivas.

La frecuencia puede expresarse aproximadamente mediante:

$$f = \frac{1}{NT_s} \quad (3)$$

donde:

- N representa la cantidad de muestras.
- T_s representa el período entre muestras.

El período de muestreo se controla utilizando un temporizador del ATmega328P.

V-F. Código

El acceso a la tabla de valores digitales (TABLA8) se realiza mediante el registro puntero indirecto con desplazamiento Z (R31:R30). Dado que las tablas de datos están almacenadas en memoria de programa (Flash) utilizando la directiva `.db`, se emplea la instrucción `lpm` (Load Program Memory) para recuperar los valores. El índice de la tabla se incrementa en cada interrupción, permitiendo recorrer secuencialmente los 128 puntos que conforman el ciclo de la señal.

Adicionalmente, se implementa un sistema de menú serial que permite al usuario seleccionar la señal (8 o 17) mediante el registro `seleccion`, y ajustar la velocidad de actualización mediante los comandos `+` y `-`. El registro `temp` se utiliza para almacenar temporalmente el carácter recibido por la UART y validar la entrada del usuario, descartando caracteres de control como retorno de carro (`\n`) y nueva línea (`\r`).

Listing 2. Ejemplo de lectura de una LUT

```

1
2 MAIN:
3     rcall uart_rx
4     cpi temp, 13
5     breq MAIN
6     cpi temp, 10
7     breq MAIN
8     cpi temp, '1'
9     breq CMD_SENAL8
10    cpi temp, '2'
11    breq CMD_SENAL17
12    cpi temp, '+'
13    breq CMD_RAPIDO
14    cpi temp, '-'
15    breq CMD_LENTO
16    rjmp MAIN
17 ;
18 CMD_SENAL8:
19     cli
20     ldi seleccion, 8
21     ldi indice, 0
22     sei
23     ldi ZL, LOW(msg_senal8<<1)
24     ldi ZH, HIGH(msg_senal8<<1)
25     rcall uart_print
26     rjmp MAIN
27 ;
28 USAR_SENAL8:
29     cpi indice, 128
30     brlo SENAL8_OK
31     ldi indice, 0
32 ;

```

```

33 SENAL8_OK:
34     ldi ZL, LOW(TABLA8<<1)
35     ldi ZH, HIGH(TABLA8<<1)
36     add ZL, indice
37     clr temp
38     adc ZH, temp
39     lpm dato, Z
40     rcall DAC_ESCRIBIR
41     inc indice
42     rjmp FIN_ISR
43 ;
44 TABLA8:
45     .db 112,112,112,112,112,112,112,112
46     .db 112,112,112,112,112,112,112,112
47     ...
48

```

VI. PROBLEMA C: CONTROL DEL PLOTTER

VI-A. Descripción

El Problema C consiste en programar el microcontrolador ATmega328P para controlar un plotter mediante señales enviadas a un Controlador Lógico Programable (PLC).

La solución consta del dibujo de diversas figuras mediante un único comando: un triángulo, un círculo, un pentagrama, una 'X' (figura a elección) y un Pokémon (en nuestro caso, un Voltorb). Para ejecutar cada figura, se selecciona su letra correspondiente en el menú, y se incluyó una opción final que permite dibujar todas simultáneamente.

VI-B. Materiales utilizados

VI-C. Conexiones

Las conexiones utilizadas son mostradas en la Tabla I.

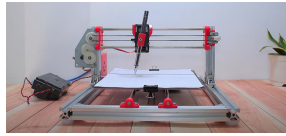
Cuadro I
CONEXIONES ENTRE EL MICROCONTROLADOR Y EL PLOTTER.

Pin	Función
D2	Bajar solenoide X0
D3	Subir solenoide X1
D4	Movimiento abajo X5
D5	Movimiento arriba X6
D6	Movimiento derecha X7
D7	Movimiento izquierda X10

VI-D. Opciones disponibles

El usuario puede seleccionar mediante comunicación serial:

- 1: Triángulo.
- 2: Círculo.
- 3: Pentagrama.
- 4: Cruz (Figura libre).
- P: Pokémon asignado.
- T: Todas las figuras.



Plotter

Figura 6. Materiales utilizados en el Problema C.

VI-E. Máquina de estados



Figura 7. Máquina de estados simplificada del plotter.

VI-F. Control de movimientos

Cada desplazamiento se obtiene activando una salida durante un determinado período de tiempo.

```

23 out PORTD, r16
24 call delay
25 ret
26 SD:
27 ;Movimiento hacia abajo E:
28 ldi r16, 0b01010100
29 out PORTD, r16
30 call delay
31 ret
32 WD:
33 ;Movimiento hacia abajo E:
34 ldi r16, 0b01100100
35 out PORTD, r16
36 call delay
37 ret
38 A:
39 ;Movimiento hacia izquierda E:
40 ldi r16, 0b10000100
41 out PORTD, r16
42 call delay
43 ret
44 D:
45 ;Movimiento hacia derecha E:
46 ldi r16, 0b01000100
47 out PORTD, r16
48 call delay
49 ret
50 W:
51 ;Movimiento hacia la arriba E:
52 ldi r16, 0b00100100
53 out PORTD, r16
54 call delay
55 ret
56 S:
57 ;Movimiento hacia la abajo E:
58 ldi r16, 0b00010100
59 out PORTD, r16
60 call delay
61 ret
  
```

Listing 3. Ejemplo de control de movimientos

```

1 ;Comando del plotter
2 E:
3 ;Bajar solenoide: ; D2
4 ldi r16, 0b00000100
5 out PORTD, r16
6 call delay
7 ret
8 Q:
9 ;Subir solenoide: ; D3
10 ldi r16, 0b00001000
11 out PORTD, r16
12 call delay
13 ret
14 WA:
15 ;Movimiento hacia arriba E:
16 ldi r16, 0b10100100
17 out PORTD, r16
18 call delay
19 ret
20 SA:
21 ;Movimiento hacia abajo E:
22 ldi r16, 0b10010100
  
```

VII. PROBLEMA D: COMUNICACIÓN USART-UART

VII-A. Descripción

En este problema se utilizan dos microcontroladores ATmega328P conectados mediante comunicación USART.

El primer microcontrolador funciona como transmisor y posee tres pulsadores.

Cada pulsador representa un bit, permitiendo ingresar un número binario de tres bits:

$$000_2 \leq D \leq 111_2$$

(4)

equivalente a:

$$0 \leq D \leq 7$$

(5)

El valor seleccionado se transmite mediante USART al segundo microcontrolador.

El receptor interpreta el dato y activa la salida LED correspondiente.

VII-B. Materiales utilizados

VII-C. Ejemplo de funcionamiento

Si los pulsadores presentan el valor:

$$011_2$$

(6)

su equivalente decimal es:

$$011_2 = 3_{10}$$

(7)

Por lo tanto, el segundo microcontrolador activa la salida correspondiente al valor 3.

VII-D. Diagrama general

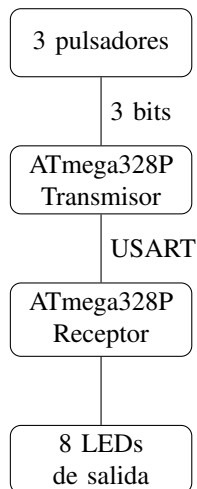


Figura 9. Comunicación USART entre los dos microcontroladores.

VII-E. Funcionamiento del transmisor

El transmisor realiza las siguientes acciones:

1. Lee el estado de los tres pulsadores.
2. Forma un dato binario de tres bits.
3. Transmite el valor mediante USART.
4. Repite continuamente el proceso.

VII-F. Funcionamiento del receptor

El receptor:

1. Espera la recepción del dato.
2. Lee el registro USART.
3. Verifica que el valor esté entre 0 y 7.
4. Decodifica el dato.
5. Activa la salida correspondiente.

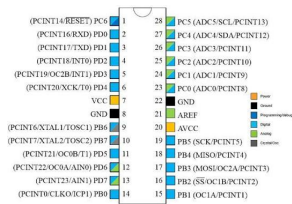
VII-G. Código del transmisor

Listing 4. Estructura del transmisor USART

```

1
2 .include "m328Pdef.inc"
3
4 .cseg
5
6 .org 0x0000
7
8     rjmp INICIO
9
10
11 INICIO:
12
13     ; Stack Pointer
14
15     ldi r16, HIGH(RAMEND)
16     out SPH, r16
17
18     ldi r16, LOW(RAMEND)
19     out SPL, r16
20
21     ; Configurar entradas
22     ; Configurar USART
23
24
25 LOOP:
26
27     ; Leer pulsadores
28
29     in r16, PINB
30
31     ; Conservar 3 bits
32
33     andi r16, 0x07
34
35     ; Transmitir
36
37     rcall USART_TX
38
39     rjmp LOOP
40
41
42 USART_TX:
43
44 ESPERAR_TX:
45
46     lds r17, UCSRA
47
48     sbrc r17, UDRE0
49
50     rjmp ESPERAR_TX
51
52     sts UDR0, r16
53
54     ret

```



ATmega328P



Pulsadores



LEDs



Cables

Figura 8. Materiales utilizados en el Problema D.

VII-H. Código del receptor

Listing 5. Estructura del receptor USART

```

1  .include "m328Pdef.inc"
2
3
4  .cseg
5
6  .org 0x0000
7
8  rjmp INICIO
9
10 INICIO:
11
12     ; Inicializacion
13     ; Configuración USART
14     ; Configuración de LEDs
15
16
17
18 LOOP:
19
20     rcall USART_RX
21
22     ; Valor recibido en r16
23
24     rcall ACTIVAR_LED
25
26     rjmp LOOP
27
28
29 USART_RX:
30
31 ESPERAR_RX:
32
33     lds r17, UCSR0A
34
35     sbrc r17, RXC0
36
37     rjmp ESPERAR_RX
38
39     lds r16, UDR0
40
41     ret

```

VIII. PROBLEMA E: AUTOMATIZACIÓN DE UNA PUERTA DE GARAJE

VIII-A. Descripción

El Problema E consiste en controlar automáticamente una puerta de garaje mediante un ATmega328P.

El sistema dispone de:

- Pulsador de apertura.
- Pulsador de cierre.
- Sensor de puerta abierta.

- Sensor de puerta cerrada.
- Motor de apertura y cierre.
- Alarma sonora.
- Sensor de obstáculos.
- Comunicación USART.

Durante el desplazamiento de la puerta se activa una alarma sonora.

Cuando se detecta un obstáculo, el sistema debe detener inmediatamente el movimiento y apagar la alarma.

VIII-B. Materiales utilizados

VIII-C. Máquina de estados

Para controlar la puerta se utilizan cinco estados:

- S_0 : Puerta cerrada.
- S_1 : Abriendo.
- S_2 : Puerta abierta.
- S_3 : Cerrando.
- S_4 : Detenida por seguridad.

VIII-D. Estado puerta cerrada

Cuando el sistema se encuentra en el estado de puerta cerrada:

- El motor permanece detenido.
- La alarma está apagada.
- El sistema espera la orden de apertura.

Cuando se presiona el pulsador de apertura se realiza la transición al estado *Abriendo*.

VIII-E. Estado abriendo

En este estado:

- Se activa el motor en sentido de apertura.
- Se activa la alarma.
- Se verifica el sensor de puerta abierta.

Cuando el sensor detecta que la puerta está completamente abierta, el motor se detiene y el sistema pasa al estado *Puerta abierta*.

VIII-F. Estado puerta abierta

El motor permanece detenido y la alarma apagada.

El sistema espera una solicitud de cierre.

Al presionar el pulsador correspondiente se realiza la transición al estado *Cerrando*.



Motor



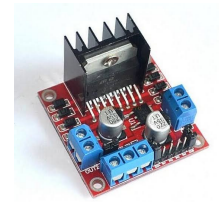
Alarma sonora



Pulsadores

(PCINT14:RESET) PC6	1	PC5 (ADC5/SCL/PCINT13)	28
(PCINT16:RXD) PD0	2	PC4 (ADC4/SDA/PCINT12)	27
(PCINT17:TXD) PD1	3	PC3 (ADC3/PCINT11)	26
(PCINT18:INT0) PD2	4	PC2 (ADC2/PCINT10)	25
(PCINT19:OC2B/INT1) PD3	5	PC1 (ADC1/PCINT9)	24
(PCINT20:OC2B/INT0) PD4	6	PC0 (ADC0/PCINT8)	23
VCC	7	GND	22
GND	8	AREF	21
(PCINT0:XTAL1/TOSC1) PB9	9	AVCC	20
(PCINT0:XTAL2/TOSC2) PB8	10	PB5 (SCK/PCINT5)	19
(PCINT21:OC0B/T1) PD5	11	PB4 (MISO/PCINT4)	18
(PCINT22:OC0A/AIN0) PD6	12	PB3 (MOSI/OC2A/PCINT3)	17
(PCINT23:AIN1) PD7	13	PB2 (SS/OC1B/PCINT2)	16
(PCINT0:CLKO/ICP1) PB9	14	PB1 (OC1A/PCINT1)	15

ATmega328P



Puente H

Figura 10. Materiales utilizados en el Problema E.

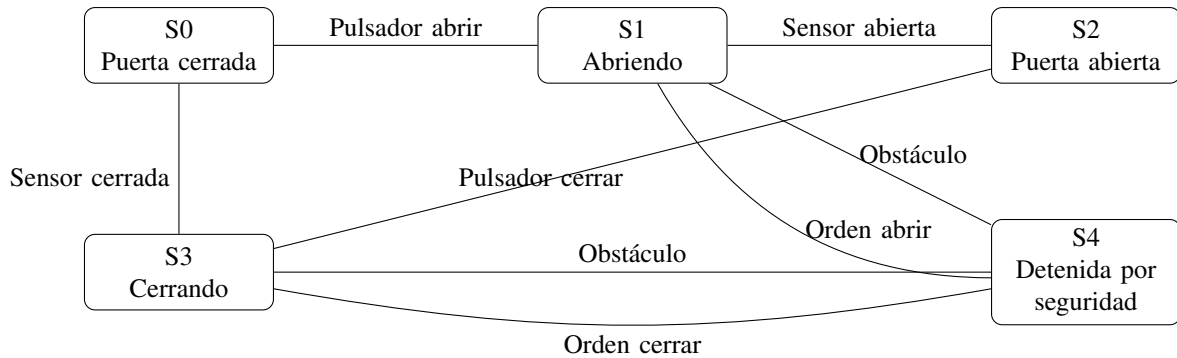


Figura 11. Máquina de estados de la puerta de garaje.

VIII-G. Estado cerrando

Durante el cierre se activa el motor en sentido contrario y se mantiene activa la alarma.

Cuando el sensor indica que la puerta está completamente cerrada, el motor se detiene y el sistema regresa al estado inicial.

VIII-H. Detección de obstáculos

El sensor de obstáculos se conecta a una entrada capaz de producir una interrupción.

Si se detecta un obstáculo durante el movimiento:

1. Se detiene inmediatamente el motor.
2. Se apaga la alarma.

3. Se cambia el estado del sistema.
4. Se informa el evento mediante USART.

VIII-I. Rutina de interrupción

Listing 6. Estructura de la interrupción de seguridad

```

1  ISR_OBSTACULO:
2
3
4  push r16
5
6  ; Detener motor
7
8  cbi PORTD, 4
9  cbi PORTD, 5
10
11 ; Apagar alarma
  
```

```

12
13 cbi PORTB, 0
14
15 ; Estado de seguridad
16
17 ldi r16, 4
18 sts ESTADO, r16
19
20 ; Mensaje mediante USART
21
22 rcall MENSAJE_OBSTACULO
23
24 pop r16
25
26 reti

```

VIII-J. Implementación de la máquina de estados

Listing 7. Estructura principal de máquina de estados

```

1 LOOP:
2
3 lds r16, ESTADO
4
5 cpi r16, 0
6 breq PUERTA_CERRADA
7
8 cpi r16, 1
9 breq ABRIENDO
10
11 cpi r16, 2
12 breq PUERTA_ABIERTA
13
14 cpi r16, 3
15 breq CERRANDO
16
17 cpi r16, 4
18 breq SEGURIDAD
19
20
21 rjmp LOOP

```

IX. CONFIGURACIÓN USART

La comunicación USART se utiliza en diferentes problemas del laboratorio.

La velocidad de comunicación se determina mediante los registros UBRR.

Para el modo asíncrono normal puede utilizarse:

$$UBRR = \frac{F_{CPU}}{16 \times Baud - 1} \quad (8)$$

donde F_{CPU} es la frecuencia del reloj del microcontrolador y $Baud$ representa la velocidad de transmisión.

IX-A. Registros principales

Cuadro II
REGISTROS UTILIZADOS POR USART.

Registro	Función
UBRR0H/L	Velocidad USART
UCSR0A	Estado del USART
UCSR0B	Habilitación TX/RX
UCSR0C	Formato de trama
UDR0	Registro de datos

X. USO DE REGISTROS DEL ATMEGA328P

Durante la implementación se utilizaron distintos registros de propósito general.

Cuadro III
EJEMPLO DE UTILIZACIÓN DE REGISTROS.

Registro	Uso
r16	Registro temporal
r17	Lectura de estados
r18	Estado actual
r19	Contadores
r20	Datos USART
r21	Variables auxiliares
Z	Acceso a tablas LUT

El uso organizado de los registros permite reducir accesos innecesarios a memoria y facilita las operaciones de comparación, temporización y control.

XI. INICIALIZACIÓN DEL STACK POINTER

Debido al uso de subrutinas e interrupciones, el Stack Pointer se configura al inicio del programa.

Listing 8. Inicialización del Stack Pointer

```

1 ldi r16, HIGH(RAMEND)
2
3 out SPH, r16
4
5 ldi r16, LOW(RAMEND)
6
7 out SPL, r16
8

```

El Stack permite almacenar las direcciones de retorno utilizadas durante las llamadas a subrutinas y las interrupciones.

También se utilizan las instrucciones PUSH y POP para guardar temporalmente el contenido de los registros.

XII. RESULTADOS

Las pruebas realizadas permitieron verificar el funcionamiento de las diferentes aplicaciones, evaluadas primero en un entorno virtual y posteriormente en hardware real.

XII-A. Simulaciones

Antes de realizar la implementación física se simuló los diferentes circuitos para verificar estados, temporizaciones, interrupciones y funcionamiento de la UART.

XII-A1. Parte A: Matriz de LEDs: En el Problema A se logró visualizar un mensaje desplazable en la matriz de LEDs y seleccionar las diferentes opciones de figuras mediante comunicación UART, también se implementó control de velocidad de desplazamiento mediante comandos seriales + -, permitiendo ajustar la frecuencia del mensaje.

XII-A2. Parte B: Convertidor DAC R-2R: Se generaron los valores correspondientes a una señal mediante una tabla LUT y un convertidor DAC R-2R. La salida analógica obtenida pudo visualizarse de forma fluida utilizando un osciloscopio virtual. Posteriormente, al realizarlo así, se tuvo que agregar una función de ajuste de frecuencia que pueda aumentar y disminuir la frecuencia de nuestras señales(17,8)

XII-A3. Parte C: Control del Plotter: Se implementaron y validaron las secuencias necesarias para controlar los movimientos del plotter y ejecutar diferentes figuras (triángulo, círculo, pentagrama, cruz y el pokémon Voltorb). Un fallo a destacar para entender correctamente los comandos, ahí que invertimos izquierda y derecha porque desde nuestro punto de vista esta al revés.

XII-A4. Parte D: Comunicación USART-UART: Se estableció una comunicación USART entre dos microcontroladores ATmega328P. El dato de tres bits generado mediante los pulsadores del transmisor fue recibido y decodificado de manera inmediata por el receptor. Se agregó un cuarto botón para el reinicio total, dado que al no presionar ningún botón, en nuestra lógica es 0; por ende, el primer LED, una vez que apretamos los botones que representan el binario 001, prendería el segundo LED, que corresponde al 1, ya que vamos de 0 a 7.

XII-A5. Parte E: Puerta de Garaje: Se simuló una máquina de estados capaz de controlar la apertura y cierre de la puerta. El sistema respondió correctamente a los sensores de posición y detuvo el motor automáticamente al activar la interrupción por detección de obstáculos, reportando el evento mediante mensaje serial correspondiente al estado actual.

XII-B. Implementación física

Luego de comprobar el funcionamiento mediante simulación, los circuitos fueron implementados físicamente en protoboard. Antes de alimentar los sistemas se verificaron las conexiones VCC/GND, la alimentación del ATmega328P y los niveles de tensión de los periféricos.

El comportamiento del hardware real coincidió con los resultados obtenidos durante la etapa de simulación para cada uno de los problemas planteados, confirmando la validez del código ensamblador desarrollado.

XII-C. Conclusiones

El desarrollo del laboratorio permitió profundizar en el uso de periféricos del ATmega328P mediante programación en ensamblador, verificando la importancia de comprender el funcionamiento interno de registros y banderas para implementar soluciones eficientes.

La metodología de simular antes de implementar físicamente redujo significativamente los errores de conexión y

temporización en los problemas A, B, D y E. En el caso particular del Problema C (plotter), la naturaleza mecánica del sistema y su acoplamiento con el PLC hizo más eficiente abordar el desarrollo directamente sobre hardware mediante iteración experimental, ajustando tiempos de retardo e invirtiendo lógicamente los ejes X e Y hasta obtener las figuras correctas. Esta diferencia de enfoque entre problemas evidenció que no todos los sistemas se benefician igualmente de la simulación previa, depende mucho del caso.

Las máquinas de estados demostraron ser una herramienta fundamental para estructurar sistemas con múltiples modos de operación, particularmente en el problema de la puerta de garaje donde la interrupción por obstáculo requirió coordinación precisa entre rutina de servicio y estado global del sistema.

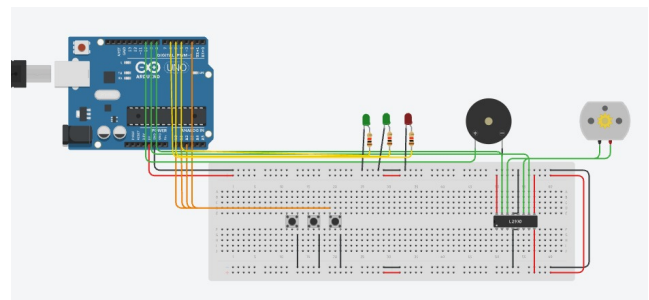


Figura 12. Simulación del sistema en Tinkercad Parte E.

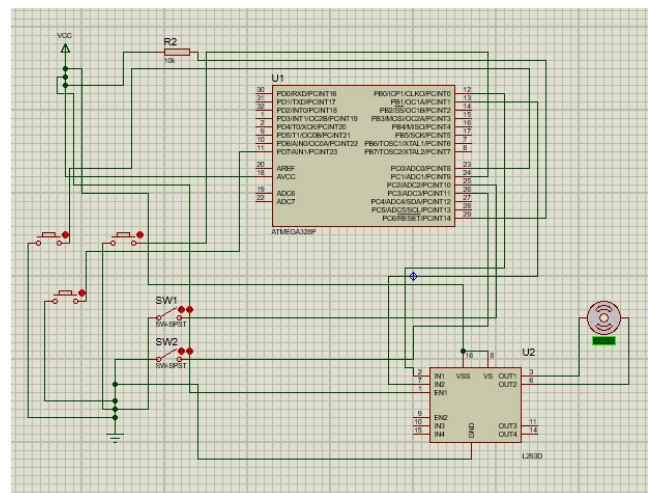


Figura 13. Simulación del sistema en Proteus Parte E.

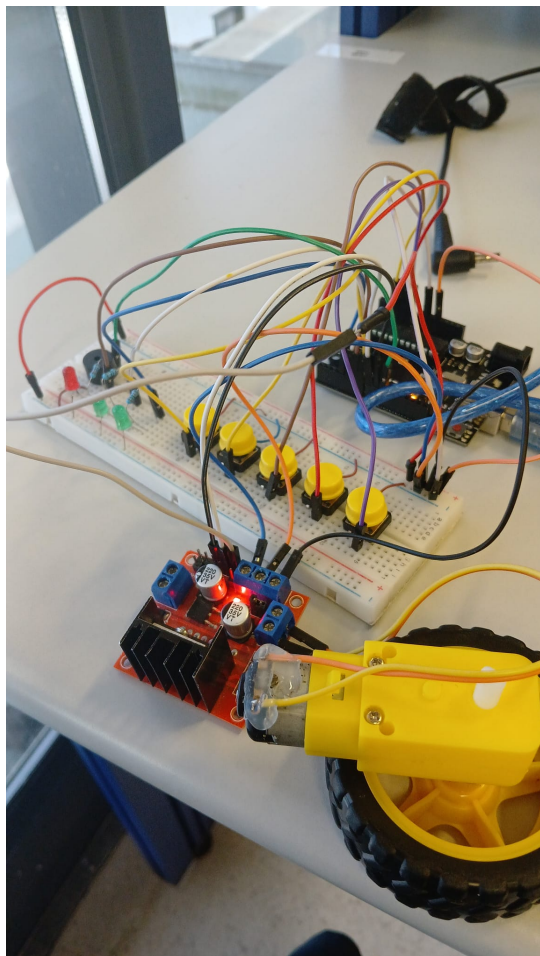


Figura 14. Implementación física del sistema Parte E.

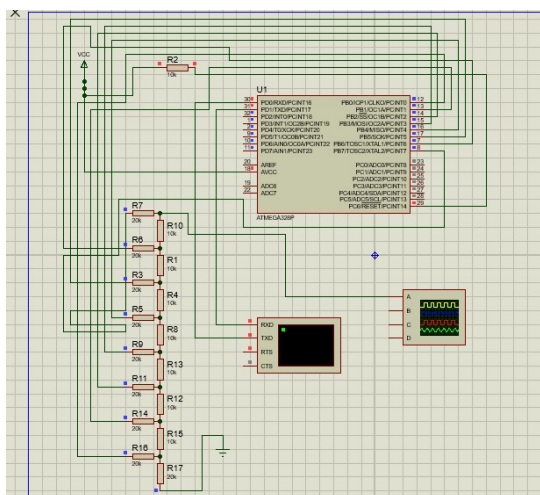


Figura 15. Simulación en Proteus Parte B.

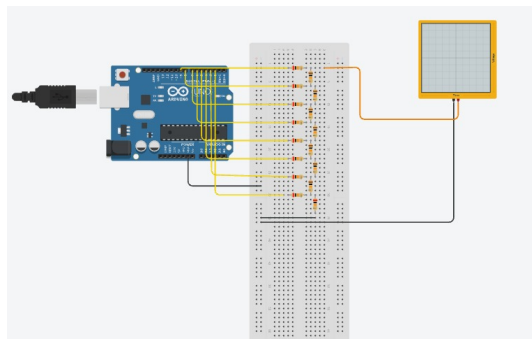


Figura 16. Simulación En tinkercard parte B.

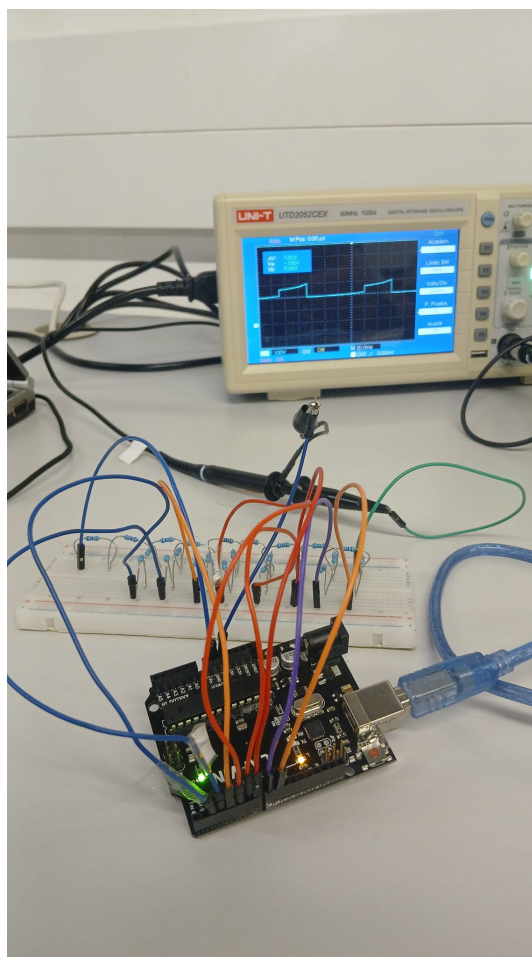


Figura 17. Montaje físico parte B.

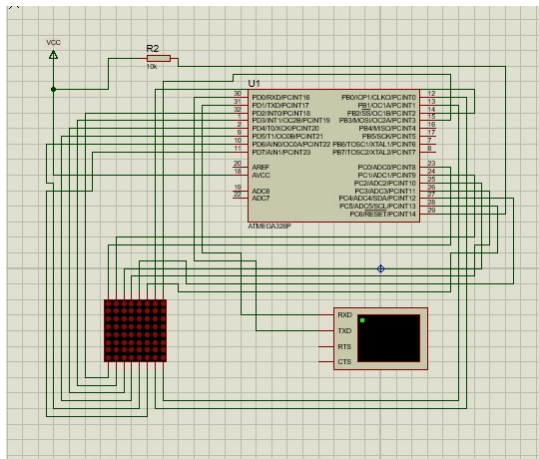


Figura 18. Simulacion en Proteus parte A.

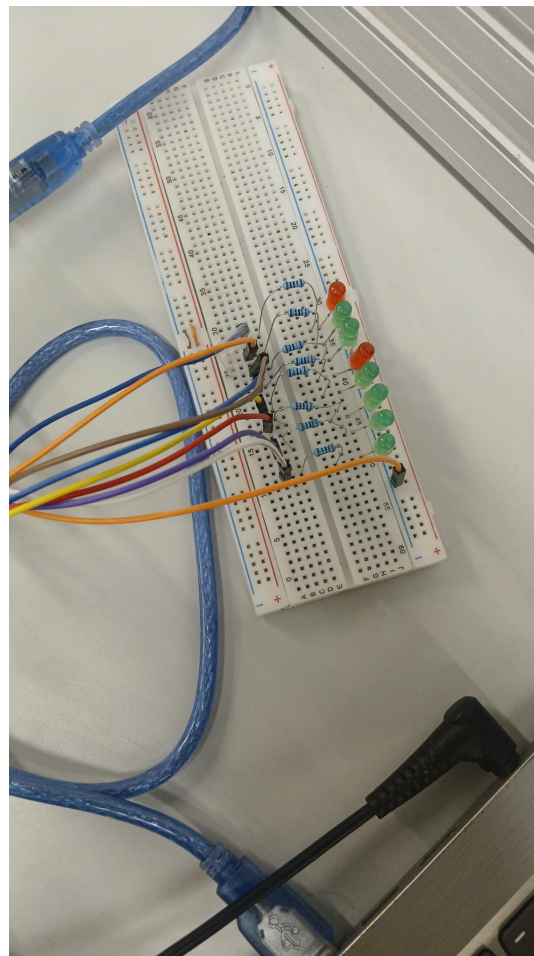


Figura 20. Implementacion fisica parte D UART 2.

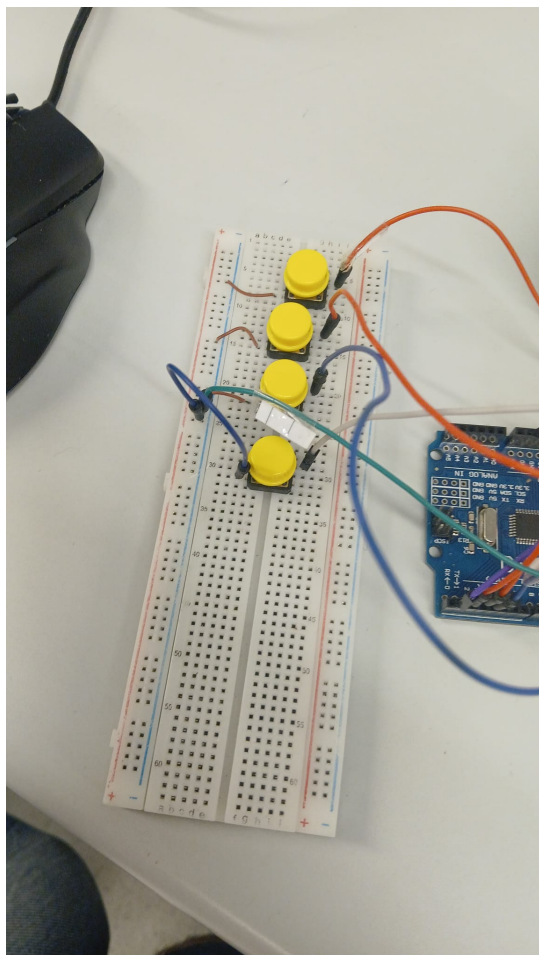


Figura 19. Implementacion fisica parte D UART1.

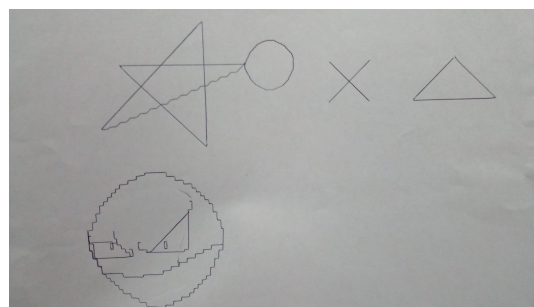


Figura 21. Resultado del Plotter parte C.

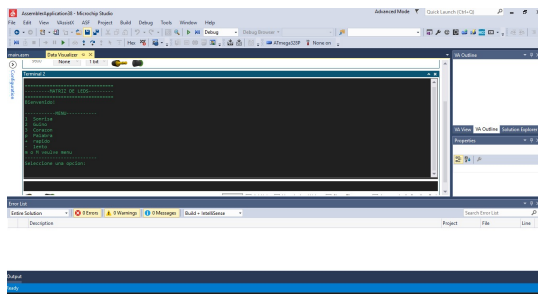


Figura 22. Menu de la parte A.

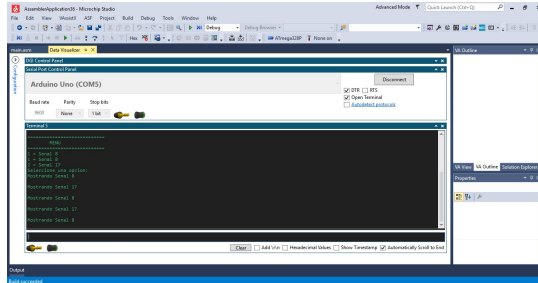


Figura 23. Menu de la parte B.

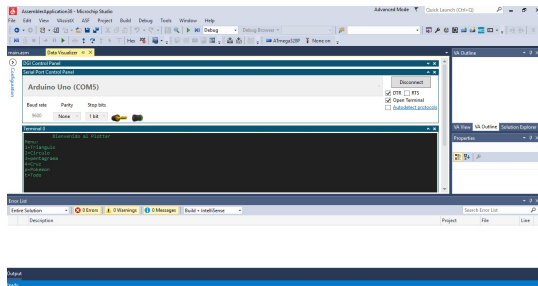


Figura 24. Menu de la parte C.

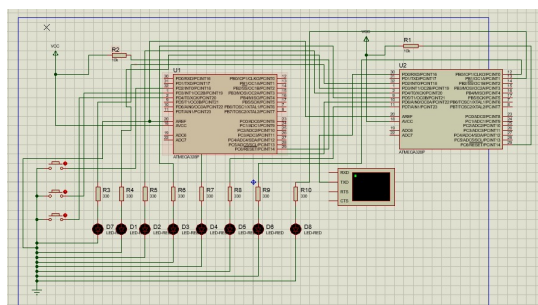


Figura 25. Simulacion en Proteus de la parte D.

XIII. CONTROL DE VERSIONES Y REPOSITORIO GITHUB

El código desarrollado durante el laboratorio fue administrado mediante GitHub.

El repositorio utilizado se encuentra disponible en:

[https:](https://github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio2)

[//github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio2](https://github.com/Braian-Arias/Tec.Micro/tree/main/Laboratorio2)